Flash Open Source Solution Developer Guide

ID: RK-KF-YF-314

Release Version: V3.15.0

Release Date: 2025-05-11

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

Trademark Statement

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

In the SDK with kernel version 4.4 and older, the driver code of RK small capacity storage is self-developed code. Considering that the support of the open source community for parallel port Nand, SPI Nand and SPI Nor has been gradually improved in recent years, and the maintenance of UBIFS is relatively stable, the SDK with kernel version 4.4 will gradually replace the storage support with the open source solution, The SDK with kernel version 5.10 has been converted to full open source support.

Product Version

Chipset	Kernel version
All SOC support FSPI(SFC)	Linux 4.4, Linux4.19, Linux 5.10, Linux 6.1

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	HKH	2019-06-20	Initial version
V1.0.1	HKH	2019-11-11	Add SD card upgrade introduction
V1.0.2	Ruby Zhang	2020-07-08	Update the format of the document
V1.1.0	Jair Wu	2020-07-10	Add u-boot compile introduction
V2.0.0	Jon Lin	2020-10-19	Improve the driver configuration and other details
V2.0.1	Jon Lin	2020-11-27	Add UBIFS multi volume support, increase or decrease ubiattach parameter description
V2.1.0	Jon Lin	2021-01-27	Add more UBIFS support
V2.1.1	CWW	2021-02-22	Update the format of the document
V2.2.0	Jon Lin	2021-04-13	Supports u-boot UBIFS and change to use programmer_image_tool
V2.3.0	Jon Lin	2021-07-06	Add IDB layout description, optimize vendor partition description and ECC related description
V2.4.0	Jon Lin	2021-10-29	Support Linux 5.10, Add more details
V2.5.0	Jon Lin	2022-02-28	Upgrade the OTA strategy of SLC Nand
V3.0.0	Jon Lin	2022-03-06	Remove RK3308 patches, Support SPI Nor, Add GPT partition extension concept
V3.1.0	Jon Lin	2022-07-28	Optimize document structure, Introduce SPI Nor OTA by shell command
V3.2.0	Jon Lin	2023-06-05	Detail the driver configuration
V3.3.0	Jon Lin	2023-08-15	Add SPI No ENV scheme burner burning chapter and add testing items
V3.4.0	Jon Lin	2023-12-04	Add SPI Flash Kernel speed calculation and explanation of Nand product lifespan issues

Version	Author	Date	Change Description
V3.5.0	Jon Lin	2024-01-08	Add explanation for abnormal power lost issues
V3.6.0	Jon Lin	2024-01-10	Add signal testing instructions and more common questions
V3.7.0	Jon Lin	2024-01-18	Add chip and corresponding software solution information, and add partition table description information
V3.8.0	Jon Lin	2024-03-04	Add support for RK3576 and Linux 6.1 kernel version
V3.9.0	Jon Lin	2024-07-25	Add support for RV1103B/RK3506
V3.10.0	Jon Lin	2024-08-19	Fix SPI Nor 4KB sector erase JFFS2 support instructions
V3.11.0	Jon Lin	2024-09-05	Add RV1106B
V3.12.0	Jon Lin	2024-09-09	Restrict SPI Nor 4KB erase granularity for JFFS2 application scope.
V3.13.0	Jon Lin	2024-11-01	Optimize Nand power-off exception plan, add an introduction to UBI FASTMAP.
V3.15.0	Jon Lin	2025-05-11	Fix SPI Nand MTD command aligned size and add mtd read operation return result note.

Contents

1. Notes on Open Source Solution

1.1 Open Source Solution Feature

Confirm these following feature:

Note.	Support Device Type	Register Device Type	Filesystem	Download methods
SLC Nand open source solution (Parallel Nand)	SLC Nand	mtd, ubiblock	SquashFS, UBIFS	USB download, SD card download, Flash Programmer
SPI Nand open source solution	SPI Nand	mtd, ubiblock	SquashFS, UBIFS	USB download, SD card download, Flash Programmer
SPI Nor open source solution	SPI Nor	mtd, mtd block	SquashFS, JFFS2	USB download, SD card download, Flash Programmer

Main points:

- The device type to choose
- Does the filesystem meet the requirements

1.2 Notes on NAND flash selection

Because UBIFS relies on the specific specifications of Nand equipment for production, the produced UBI image cannot be compatible with Nand of different page size and block size.

- Nand material selection specifications shall be consistent.
- SLC Nand in the document refers to the parallel port NAND

1.3 SOC Corresponding Partition Extension

As described in the "Partition Configuration" chapter, the RK SDK has made some special extensions for MTD partitions. Please confirm the extension support according to the chip:

SOC	GPT Partition Extension	ENV Partition Extension
RK3308	Y	N
RV1126 & RV1109	Y	N
RK3568 & RK3566	Y	N
RK3588 & RK3588s	Y (default)	Y (option)
RV1106/RV1103	N	Y
RK3528	Y	N
RK3562	Y	N
RK3576	Y	N
RV1106B/RV1103B	N	Y
RK3506	Y	N

1.4 Key Features of FSPI/SFC in Chips

Only SOC chips using open-source storage are counted:

SOC	FSPI/SFC IO rate	FSPI/SFC data io lines
RK3308	<= 48MHz (recommended)	x1\x4
RV1126 & RV1109	<= 125MHz	x1\x4
RK3568/RK3566	<= 150MHz, fixed frequency points OSC50\75\100\125\150 MHz	x1\x4
RK3588/RK3588s	<= 150MHz	x1\x4
RV1106/RV1103	<= 150MHz, Supports 125MHz and 150MHz, but due to frequency division restrictions, the frequency points between them are not supported.	x1\x4
RK3528	<= 150MHz	x1\x4
RK3562	<= 150MHz	x1\x4
RK3576	<= 133MHz, due to frequency division restrictions, the frequency points limit to 125MHz	x1\x4
RV1106B/RV1103B	FSPI0 <= 133MHz, FSPI1 <= 125MHz	x1\x4
RK3506	<= 133MHz	x1\x4

Note:

- IP supports x4 and is also compatible with x2, but the open-source framework does not support dual SPI.
- IP supports higher frequencies, but the default configuration- IP supports higher frequencies, but the default configuration of SDK is usually lower frequency, recommended at 80MHz, which can be adjusted according to the actual supported speed of the project and external particles.
- The upper limit of the rate for Quad SPI Nand is usually 133MHz, and for Quad SPI Nor is also usually 133MHz (up to 166MHz at the highest).
- The upper limit of the rate for Octal SPI Flash is usually 200MHz.

1.5 Chip Partition Extension and IDB Planning

1.5.1 Chip Partition Extension Plans

Chip	GPT Partition Extension	ENV Partition Extension
RK3308	Default plan (Compatibility plan 1)	N
RV1126/RV1109	Default plan (Compatibility plan 2)	N
RK3568/RK3566	Default plan	N
RK3588/RK3588s	Default plan	N
RV1106/RV1103	N	Default plan
RK3528	Default plan	N
RK3562	Default plan	N
RK3576	Default plan	N
RV1106B/RV1103B	N	Default plan
RK3506	Default plan	N

Note:

- For details on the modifications to the compatibility plan, please refer to the description of "GPT Partition Extension IDB Planning".

1.5.2 ENV Partition Extension IDB Planning

Storage Type	idb Start Address	Multi-backup Explanation
SLC Nand	256KB	Requires dual image backups
SPI Nand	256KB	Requires multiple image backups
SPI Nor	64KB	Requires dual image backups

1.5.3 GPT Partition Extension IDB Planning

Storage Type	idb Start Address	Multi-backup Explanation
SLC Nand	flash block size (128KB or 256KB)	Tools support dual backups, aligned with flash block size
SPI Nand	flash block size (128KB or 256KB)	Tools support dual backups, aligned with flash block size
SPI Nor	64KB	Tools support dual backups, aligned with 64KB

Note:

- The PC upgrade tool supports dual backup burning of IDB.
- Compatibility plan 1: For the RK3308 plan, SPI Nor idb only supports single backups, and blocks 1~8 of SPI Nand are idb multi-backup areas, storing up to five copies.
- Compatibility plan 2: For the RV1126/RV1109 plan, the starting address of SPI Nor idb is at 32KB, supporting only single backups.

2. Build Configuration Changes

2.1 SPL & U-Boot Basic Configuration

Configuration:

```
// MTD support
CONFIG_MTD=y
CONFIG_CMD_MTD_BLK=y
CONFIG_SPL_MTD_SUPPORT=y
CONFIG_MTD_BLK=y
CONFIG_MTD_DEVICE=y

// spi nand support
CONFIG_MTD_SPI_NAND=y
CONFIG_ROCKCHIP_SFC=y
CONFIG_SPL_SPI_FLASH_SUPPORT=y
CONFIG_SPL_SPI_SUPPORT=y

// nand support
CONFIG_NAND=y
CONFIG_CMD_NAND=y
CONFIG_NAND_ROCKCHIP=y /* NandC v6, according to TRM NANDC->NANDC_NANDC_VER register,
0x00000801 */
```

```
//CONFIG_NAND_ROCKCHIP_V9=y  /* NandC v9 according to TRM NANDC->NANDC_NANDC_VER register,
0x56393030 */
CONFIG_SPL_NAND_SUPPORT=y
CONFIG_SYS_NAND_U_BOOT_LOCATIONS=y

// spi nor support
CONFIG_CMD_SF=y
CONFIG_CMD_SPI=y
CONFIG_SPI_FLASH=y
CONFIG_SF_DEFAULT_MODE=0x1
CONFIG_SF_DEFAULT_SPEED=50000000
CONFIG_SPI_FLASH_GIGADEVICE=y
CONFIG_SPI_FLASH_MACRONIX=y
CONFIG_SPI_FLASH_WINBOND=y
CONFIG_SPI_FLASH_MTD=y
CONFIG_ROCKCHIP_SFC=y
CONFIG_SPL_SPI_SUPPORT=y
CONFIG_SPL_MTD_SUPPORT=y
CONFIG_SPL_SPI_FLASH_SUPPORT=y
```

Remove rkflash/rknand configuration:

```
CONFIG_RKFLASH=n
CONFIG_RKNAND=n
```

The codes:

```
./drivers/mtd/nand/raw          //SLC Nand controller and protocal layer
./drivers/mtd/nand/spi          //SPI Nand protocal layer
./drivers/spi/rockchip_sfc.c    //SPI Flash controller
./drivers/mtd/spi               //SPI Nor protocal layer
```

Explanation:

- SPI Nor flash chip, the software supports SPI Nand flash chips that can be queried from the source code `drivers/mtd/spi/spi-nor-ids.c`
- For example, with Winbound SPI Nand flash chip, the software supports SPI Nand flash chips that can be queried from the source code `drivers/mtd/nand/spi/winbond.c`, and other chips are supported in the same directory level
- You can match the source code by flash id or flash silkscreen.

2.2 Kernel Basic Configuration

2.2.1 SLC Nand Open Source Solution

The framwork:

Note.	Support Device Type	Driver Codes	Flash Framework
SLC Nand open source solution	SLC Nand	drivers/mtd/nand/raw	drivers/mtd/nand/raw

Configuration:

```
CONFIG_RK_NANDC_NAND=n   /* It's not compatible */
CONFIG_MTD_NAND_ROCKCHIP_V6=y /* NandC v6 is depending on TRM NANDC->NANDC_NANDC_VER
register, 0x00000801 */
# CONFIG_MTD_NAND_ROCKCHIP_V9=y /* NandC v9  is depending on TRM NANDC->NANDC_NANDC_VER
register, 0x56393030 */
CONFIG_MTD_CMDLINE_PARTS=y
```

2.2.2 SPI Nand Open Source Solution

2.2.2.1 Linux 4.19 and former version

The framework:

Note.	Support Device Type	Driver Codes	Flash Framework
SPI Nand open source solution	SPI Nand	drivers/rkflash	drivers/rkflash
SPI Nor open source solution	SPI Nor	drivers/rkflash	drivers/rkflash

Defconfig configuration:

```
CONFIG_RK_FLASH=y

CONFIG_RK_SFC_NAND=y     /* SPI Nand flash */
CONFIG_RK_SFC_NAND_MTD=y /* SPI Nand flash and partitions is register as mtd device,
otherwise block devices(rkflash0pn)  */
CONFIG_RK_SFC_NOR=y      /* SPI Nor flash */
CONFIG_RK_SFC_NOR_MTD=y  /* SPI Nor flash and partitions is register as mtd device,
otherwise block devices(rkflash0pn) */
CONFIG_MTD_CMDLINE_PARTS=y
```

dts configuration:

```
&sfc {
    assigned-clocks = <&cru SCLK_SFC>; # Select the clock source corresponding to clk_sfc in
the dtsi sfc device node to modify the IO rate.
    assigned-clock-rates = <100000000>;
    status = "okay";
};
```

2.2.2.2 Linux 5.10 and later versions

Note.	Support Device Type	Driver Codes	Flash Framework
SPI Nand open source solution	SPI Nand	drivers/spi/spi-rockchip-sfc.c	drivers/mtd/nand/spi
SPI Nor open source solution	SPI Nor	drivers/spi/spi-rockchip-sfc.c	drivers/mtd/spi-nor

Explanation:

- For example, with Winbound SPI Nor flash chip, the software supports SPI Nor flash chips that can be queried from the source code `drivers/mtd/spi-nor/winbond.c`, and other chips are supported in the same directory level.
- For example, with Winbound SPI Nand flash chip, the software supports SPI Nand flash chips that can be queried from the source code `drivers/mtd/nand/spi/winbond.c`, and other chips are supported in the same directory level.
- You can match the source code by flash id or flash silkscreen.
- The software identifies specific flash chips through their flash ids. If the configuration of the chip in the source code and the bus-width configuration of the controller node in dts are both set to quad/octal, the driver will select the quad/octal transmission scheme.

Defconfig configuration:

```
CONFIG_SPI_ROCKCHIP_SFC=y
CONFIG_MTD_SPI_NAND=y
CONFIG_MTD_SPI_NOR=y
```

dts :

```
&sfc {
    sfc-cs-gpios = <&gpio4 RK_PB0 GPIO_ACTIVE_LOW>, <&gpio4 RK_PA5 GPIO_ACTIVE_LOW>; #
Extend gpio as CSN
    status = "okay";

    flash@0 {
        compatible = "spi-nand";    # compatible = "jedec,spi-nor"; for spinor
        reg = <0>;
        spi-max-frequency = <75000000>; # Modify IO rate
        spi-rx-bus-width = <4>;         # Confirm the supported line width by reading "Key
Features of FSPI/SFC in Chips"
        spi-tx-bus-width = <1>;         # Confirm the supported line width by reading "Key
Features of FSPI/SFC in Chips". If x4 is not supported, the software will automatically
downgrade to x1 for better compatibility
    };
};
```

2.3 Partition Configuration

2.3.1 MTD Partition Basics

Open source solution flash devices is register as MTD devices, and only support mtd partition, mtdparts partition table is usually defined in the kernel dts cmdline.

2.3.2 GPT Partition Extension

Some SDK schemes of RK support parsing GPT in u-boot and generating the information required by MTD mtdparts, which is finally transmitted to the kernel through cmdline. You can confirm the corresponding chip platform by referring to the chapter "SOC Corresponding Partition Extension", The code to convert GPT to mtdparts in u-boot is implemented in the file `u-boot/drivers/mtd_blk.c.`

The GPT partition table is also configured through a parameter file, similar in structure to MTD Partition, with four differences:

1. Set TYPE as GPT
2. No parameter partitions are defined (if defined, they will not be used)
3. The last partition needs to add the keyword "grow"
4. MTD scheme does not need to specify the uuid of rootfs

```
FIRMWARE_VER:8.1
MACHINE_MODEL:RK3326
MACHINE_ID:007
MANUFACTURER: RK3326
MAGIC: 0x5041524B
ATAG: 0x00200800
MACHINE: 3326
CHECK_MASK: 0x80
PWR_HLD: 0,0,A,0,1
TYPE: GPT
CMDLINE:mtdparts=rk29xxnand:0x00002000@0x00004000 (uboot),0x00002000@0x00006000
(trust),0x00002000@0x00008000 (misc),0x00008000@0x0000a000 (resource),0x00010000@0x00012000
(kernel),0x00010000@0x00022000 (boot),0x00020000@0x00032000 (recovery),0x00038000@0x00052000
(backup),0x00002000@0x0008a000 (security),0x000c0000@0x0008c000
(cache),0x00300000@0x0014c000 (system),0x00008000@0x0044c000
(metadata),0x000c0000@0x00454000 (vendor),0x00040000@0x00514000 (oem),0x00000400@0x00554000
(frp),-@0x00554400 (userdata:grow)
```

GPT Partition Table Upgrade Process:

1. Tool reads partition definitions in the parameter file.
2. Obtains the storage device capacity from the loader.
3. Modifies the size of the last partition and creates a GPT partition table file.
4. Burns the partition table to the storage device at address 0 and -33 (end).

5. The parameter file itself is not burned into the storage device.

2.3.3 ENV Partition Extension

RK Partial SDK Scheme Supports Setting ENV Partition, Supports u-boot Standard ENV Environment Variables, and Passes It to the Kernel Through cmdline. You Can Confirm the Corresponding Chip Platform by Checking the "Chip Corresponding Partition Extension Support" Chapter.

Making ENV Image is Completed by the `./tools/mkenvimage` Tool in the u-boot Directory, Customized by Configuration File, such as Defining env.txt:

```
mtdparts=spi-
nand0:256K(env),256K@256K(idblock),4M(uboot),8M(boot),72M(rootfs),32M(userdata),-(media)
sys_bootargs=ubi.mtd=4 root=ubi0:rootfs rootfstype=ubifs
```

Explanation:

- SPI Nand Making Command Reference: ./tools/mkenvimage -s 0x40000 -p 0x0 -o env.img env_.txt
- SPI Nor Making Command Reference: ./tools/mkenvimage -s 0x10000 -p 0x0 -o env.img env.txt
- sys_bootargs is subject to actual SDK configuration, refer to the development manual for corresponding modifications

2.3.4 Notes for Nand Partition Definition

- The open source solution for SLC Nand and SPI Nand should reserve 3 to 4 flash block sizes of redundant space for each partition, so that when encountering bad blocks, there is redundant space that can be replaced. It is especially important to check if the u-boot partition has reserved space. If the partition size exceeds 10MB, it is recommended to increase the redundant space. The calculation method is 4 + the number of blocks occupied by the partition * 1%. For example, if the flash block size is 128KB and the oem space size is 16MB, which occupies 128 flash blocks, you can consider filling in a value of 5
- Partition should start from address which is flash block size aligned
- The last 4 blocks are reserved for Nand bad block table, so the firmware should not overlay the area, there are two specific situations:
    - If the last partition contains these blocks, it willIf the last partition contains these blocks, it will contain 4 manually marked bad blocks, equivalent to a fixed decrease of 4 flash blocks in the effective space of the partition. If the last image is much smaller than the size of the partition, it will not cause any practical problems, but it is still not recommended to include the bad block table area

2.4 Vendor Storage

Vendor Storage is designed to store some non-secure small data, such as SN, MAC, etc.

Configuration

Define the partition for the "vnvm" command:

- It is recommended to plan the vnvm partition after the IDB partition and before the uboot partition, with a size of 4 flash blocks, SPI Nor 256KB, SPI Nand 1MB (compatible with 128KB/256KB)
- If the parameter.txt adds an SPI Nor vnvm item, `0x00000200@0x00000800(vnvm)`, SPI Nand vnvm item `0x00000800@0x00001800(vnvm)`
- If the env.txt adds an SPI Nor vnvm item, `256K@1M(vnvm)`, SPI Nand vnvm item `1M@3M(vnvm)`

u-boot defconfig configuration:

When `CONFIG_MTD_BLK = y` is enabled by default, the key code `arch/arm/mach-rockchip/vendor.c` and the interface `./arch/arm/include/asm/arch-rockchip/vendor.h` are used.

kernel defconfig configuration:

The macro switch `CONFIG_ROCKCHIP_MTD_VENDOR_STORAGE=y` is used, the key code `drivers/soc/rockchip/mtd_vendor_storage.c` and the interface `./include/linux/soc/rockchip/rk_vendor_storage.h` are used.

User Tools

Support for PC tools, mass production tools, and kernel user-level interfaces is provided. For more details, please refer to the document "Rockchip_Application_Notes_Storage_CN.pdf".

3. PC Tools For Downloading

3.1 PC Tools For Downloading - GPT Partition Extension

Tools version requirements:

- AndroidTools tools version should equals V2.7.8 or above.
- upgrade_tools tools version should equals V1.5.6 or above.

Note:

- PC tool burning will automatically copy multiple copies of IDB firmware from block 1 to block 7, that is:
  - The first 1MB of page size 2KB flash is GPT partition and IDB space
  - The first 2MB of page size 4KB flash is GPT partition and IDB space
- The PC tool burning of nor device will make double backup of IDB firmware, which corresponds to the following:
  - 64KB offset for the first copy
  - The second IDB is followed by the first IDB, aligned size 64KB
- During the development process, if the GPT partition is adjusted and the firmware is upgraded, please go to the maskrom mode to upgrade, otherwise UBIFS may be mounted abnormally
- When download UBIFS images, the tools will automatically erase the whole partition, then download the images

3.2 Burning with PC Tools - Extending the ENV Partition

The SocToolKit tool is used, which supports both Windows and Linux versions, and has a corresponding mass production tool.

4. OTA

4.1 OTA Notes

- When performing online upgrade, make sure to unmount the UBIFS file system of the target partition before proceeding with the upgrade.
- For devices using MTD scheme and parameter partition information, if the product needs to upgrade the gpt partition table and idb image, the parameter.txt file should add gpt/idb partitions:
  - For block size 128KB Nand: 0x00000100@0x00000000 (gpt), 0x00000700@0x00000100 (idb)
  - For block size 256KB Nand: 0x00000200@0x00000000 (gpt), 0x00000e00@0x00000200 (idb)
  - For SPI Nor: 0x00000080@0x00000000 (gpt), 0x00000780@x00000080(idb)

4.2 OTA input source files

It's recommended to use the images which are made by programmer tools for OTA. Refer to chapter "Flash Programmer" for making programmer images.

4.3 Upgrade MTD Partitions By Shell Command

First of all, if the image in the MTD partition uses the UBIFS file system, refer to the chapter "UBIFS OTA" chapter . Therefore, the MTD partition is mainly aimed at the firmware partitions that are read-only and have no file system, such as IDB, u-boot, kernel, etc.

4.3.1 SLC Nand u-boot

nand info:

```
nand info
```

nand erase:

```
nand erase off size
```

- off: block size aligned, unit byte, only hexadecimal is supported, not included in OOB size

- size: block size aligned, unit byte, only hexadecimal is supported, not included in OOB size

nand write:

```
nand write.raw[.noverify] - addr off|partition [count]
```

- addr: memory address, only hexadecimal is supported, not included in OOB size
- off|partition: page size aligned, unit page, only hexadecimal is supported
- count: unit byte, only hexadecimal is supported, included in OOB size

nand read:

```
nand read.raw - addr off|partition [count]
```

- addr: memory address, only hexadecimal is supported, not included in OOB size
- off|partition: page size aligned, unit page, only hexadecimal is supported
- count: unit byte, only hexadecimal is supported, included in OOB size

For instance:

```
tftp 0x4000000 rootfs.img
nand erase 0x600000 0x200000                        /* Erase the whole partion before write
*/
nand write.raw 0x4000000 0x600000 0x1000
```

4.3.2 SLC Nand kernel

flash_erase:

```
flash_erase       /* for example: flash_erase /dev/mtd1 0 0 */
```

nanddump:

```
nanddump -o -n --bb=skipbad /dev/mtd3 -f /userdata/boot_read.img
```

- --bb=skipbad: Skip bad block
- -o: read data with oob
- -n: read data without ecc

nandwrite:

```
nandwrite -o -n -p /dev/mtd3 /rockchip_test/rockchip_test.sh
```

- -o: input file with oob
- -n: write without ecc

For instance:

```
flash_erase /dev/mtd4 0 0                       /* Erase the whole partion before write */
nandwrite -o -n -p /dev/mtd4 /userdata/boot.img
sync
nanddump -o -n --bb=skipbad /dev/mtd3 -f /userdata/boot_read.img
md5sum /userdata/boot_read.img ...
```

4.3.3 SPI Nand u-boot

SPI Nand unable to support nand command, cmd/mtd.c is available.

mtd erase:

```
mtd erase <name> <off> <size>
```

- name: spi-nand0 for SPI Nand mtd device
- off: block size aligned, unit byte, only hexadecimal is supported
- size: block size aligned, unit byte, only hexadecimal is supported

mtd write:

```
mtd write <name> <addr> <off> <size>
```

- name: spi-nand0 for SPI Nand mtd device
- addr: memory address, only hexadecimal is supported
- off: page size aligned, unit byte, only hexadecimal is supported
- size: page size aligned, unit byte, only hexadecimal is supported

mtd read:

```
mtd read <name> <addr> <off> <size>
```

- name: spi-nand0 for SPI Nand mtd device
- addr: memory address, only hexadecimal is supported
- off: page size aligned, unit byte, only hexadecimal is supported
- size: page size aligned, unit byte, only hexadecimal is supported

For instance:

```
tftp 0x4000000 rootfs.img
mtd erase spi-nand0 0x600000 0x200000                       /* Erase the whole partion
before write */
mtd write spi-nand0 0x4000000 0x600000 0x200000
```

4.3.4 SPI Nand kernel

flash_eraseall:

```
flash_erase       /* for example: flash_erase /dev/mtd1 0 0 */
```

nanddump:

```
nanddump -o -n --bb=skipbad /dev/mtd3 -f /userdata/boot_read.img
```

- --bb=skipbad: Skip bad block

nandwrite:

```
nandwrite -p /dev/mtd3 /rockchip_test/rockchip_test.sh
```

Take /dev/mtd4 for instance:

```
flash_erase /dev/mtd4 0 0                                    /* Erase the whole partion
before write */
nandwrite -p /dev/mtd4 /userdata/boot.img
sync
nanddump --bb=skipbad /dev/mtd3 -f /userdata/boot_read.img
md5sum /userdata/boot_read.img ...                            /* Add verification */
```

4.3.5 SPI Nor u-boot

Suggest mtd APIs.
mtd erase:

```
mtd erase <name> <off> <size>
```

- name: nor0 for SPI Nor mtd device
- off: sector size aligned, unit byte, only hexadecimal is supported
- size: sector size aligned, unit byte, only hexadecimal is supported

mtd write:

```
mtd write <name> <addr> <off> <size>
```

- name: nor0 for SPI Nor mtd device
- addr: memory address, only hexadecimal is supported
- off: byte aligned, unit byte, only hexadecimal is supported
- size: byte aligned, unit byte, only hexadecimal is supported

mtd read:

```
mtd read <name> <addr> <off> <size>
```

- name: nor0 for SPI Nor mtd device
- addr: memory address, only hexadecimal is supported
- off: byte aligned, unit byte, only hexadecimal is supported
- size: byte aligned, unit byte, only hexadecimal is supported

For instance:

```
tftp 0x4000000 rootfs.img
mtd erase nor0 0x600000 0x200000                          /* Erase first */
mtd write nor0 0x4000000 0x600000 0x200000
```

4.3.6 SPI Nor kernel

using mtd_debug APIs

4.4 Upgrade UBIFS Image By Shell Command

Consult to "UBIFS Instruction" -> "UBIFS OTA" chapter.

4.5 Upgrade MTD Partitions By APIs

First of all, if the image in the MTD partition uses the UBIFS file system, refer to the chapter "UBIFS OTA" chapter . Therefore, the MTD partition is mainly aimed at the firmware partitions that are read-only and have no file system, such as IDB, u-boot, kernel, etc.

u-boot

- Consult to drivers/mtd/nand/nand_util.c, Using those APIs with bad block management.
- For a complete write with less data (it is recommended to write less than 2KB data on each power on), you can consider using the corresponding interface of MTD to block device in RK SDK, source code drivers/mtd/mtd_blk.c, the block abstract interface has the following characteristics:
    - Regardless of the amount of data in a single write request, the flash block corresponding to the data will be erased. Therefore, for fragmented and frequent write behavior, calling this interface will affect the life of flash.

kernel

Consult to ./miscutils/nandwrite.c ./miscutils/flash_eraseall.c in the open source mtd source code of mtd-utils, Using those APIs with bad block management.

user

Consult to ./miscutils/nandwrite.c ./miscutils/flash_eraseall.c and conbined with mtd ioctrl in include/uapi/mtd/mtd-abi.h.

Other Notes

- The mtd read interface has four types of return values:
  - 0, data is normal, ECC OK.
  - -EUCLEAN, data is normal and usable, ECC has reached its maximum capability value. Consider updating the data (read data, erase block and then rewrite to update data).
  - -EBADMSG, data is abnormal, ECC error.
  - -EIO and other error codes, data is abnormal, controller and related processes are abnormal.
- Common judgment mechanism for mtd read interface for reference:

```
ret = mtd_read(mtd, from, len, &retlen, data);
if (ret < 0 && ret != -EUCLEAN) {
    return ret;
}
```

5. Filesystem

5.1 UBIFS Filesystem

5.1.1 Instruction

UBIFS is the abbreviation of unsorted block image file system. UBIFS is often used in file system support on raw NAND as one of the successor file systems of JFFS2. UBIFS processes actions with MTD equipment through UBIFS subsystem.

5.1.2 Configuration

Kernel Configuration:

```
CONFIG_MTD_UBI=y
CONFIG_UBIFS_FS=y
CONFIG_UBIFS_FS_ADVANCED_COMPR=y
CONFIG_UBIFS_FS_LZO=y /* Using lzo */
```

5.1.3 Image Making And Mounting

5.1.3.1 Image Making

Introduction For Commands

```
Usage: mkfs.ubifs [OPTIONS] target
Make a UBIFS file system image from an existing directory tree
Examples:
Build file system from directory /opt/img, writting the result in the ubifs.img file
        mkfs.ubifs -m 512 -e 128KiB -c 100 -r /opt/img ubifs.img
The same, but writting directly to an UBIFS volume
        mkfs.ubifs -r /opt/img/dev/ubi0_0
Creating an empty UBIFS filesystem on an UBIFS volume
        mkfs.ubifs/dev/ubi0_0
Options:
-r, -d, --root=DIR         build file system from directory DIR,
-m, --min-io-size=SIZE     minimum I/O unit size, NAND FLASH minimum write size, page
size,4096B or 2048B
-e, --leb-size=SIZE        logical erase block size, block size-2x (page size), If
block_size 256KB page_size 2KB then -e equals 258048, If block_size 128KB page_size 2KB then
-e equals 126976
-c, --max-leb-cnt=COUNT    maximum logical erase block count
-o, --output=FILE          output to FILE
-j, --jrn-size=SIZE        journal size
-R, --reserved=SIZE        how much space should be reserved for the super-user
-x, --compr=TYPE           compression type - "lzo", "favor_lzo", "zlib" or "none" (default:
"lzo")
-X, --favor-percent        may only be used with favor LZO compression and defines how many
percent better zlib should compress to make mkfs.ubifs use zlib instead of LZO (default 20%)
-f, --fanout=NUM           fanout NUM (default: 8)
-F, --space-fixup          file-system free space has to be fixed up on first mount(requires
kernel version 3.0 or greater)
-k, --keyhash=TYPE         key hash type - "r5" or "test" (default: "r5")
-p, --orph-lebs=COUNT      count of erase blocks for orphans (default: 1)
-D, --devtable=FILE        use device table FILE
-U, --SquashFS-uids        SquashFS owners making all files owned by root
-l, --log-lebs=COUNT       count of erase blocks for the log (used only for debugging)
-v, --verbose              verbose operation
-V, --version              display version information
-g, --debug=LEVEL          display debug information (0 - none, 1 - statistics, 2 - files, 3
- more details)
-h, --help                 display this help text
```

Process

1. Making UBIFS Images

```
mkfs.ubifs -F -d rootfs_dir -e real_value -c real_value -m real_value -v -o rootfs.ubifs
```

2. Making UBI volume

```
ubinize -o ubi.img -m 2048 -p 128KiB ubinize.cfg
```

- -p: block size.
- -m: NAND FLASH minimum write size which usually equals page size
- -o: output file

ubinize.cfg content:

```
[ubifs-volumn]
mode=ubi
image=rootfs.ubifs
vol_id=0
vol_type=dynamic
vol_alignment=1
vol_name=ubifs
vol_flags=autoresize
```

- mode=ubi: default.
- image=out/rootfs.ubifs: input file
- vol_id=0: volume ID, different volume id for different volume.
- vol_type=dynamic: static for read-only
- vol_name=ubifs: volume name
- vol_flags=autosize.

For Instance:

page size 2KB, page per block 64, block size 128KB, partition size 64MB:

```
mkfs.ubifs -F -d /path-to-it/buildroot/output/rockchip_rv1126_rv1109_spi_nand/target -e
0x1f000 -c 0x200 -m 0x800 -v -o rootfs.ubifs
ubinize -o ubi.img -m 2048 -p 128KiB ubinize.cfg
```

page size 2KB, page per block 128, block size 256KB, partition size 64MB:

```
mkfs.ubifs -F -d /path-to-it/buildroot/output/rockchip_rv1126_rv1109_spi_nand/target -e
0x3f000 -c 0x100 -m 0x800 -v -o rootfs.ubifs
ubinize -o ubi.img -m 2048 -p 256KiB ubinize.cfg
```

page size 4KB, page per block 64, block size 256KB, partition size 64MB:

```
mkfs.ubifs -F -d /path-to-it/buildroot/output/rockchip_rv1126_rv1109_spi_nand/target -e
0x3e000 -c 0x100 -m 0x1000 -v -o rootfs.ubifs
ubinize -o ubi.img -m 0x1000 -p 256KiB ubinize.cfg
```

Multi Volume Mirror Instance

Take a multi volume partition composed of page size 2KB, page per block 64, that is, block size 128KB, partition size 8MB, OEM and partition size 8MB UserData

```
mkfs.ubifs -F -d oem -e 0x1f000 -c 0x40 -m 0x800 -v -o oem.ubifs
mkfs.ubifs -F -d userdata -e 0x1f000 -c 0x40 -m 0x800 -v -o userdata.ubifs
ubinize -o oem_userdata.img -p 0x20000 -m 2048 -s 2048 -v ubinize_oem_userdata.cfg
```

Set ubize_ oem_ userdata.cfg As follows:

```
[oem-volume]
mode=ubi
image=oem.ubifs
vol_id=0
vol_size=8MiB
vol_type=dynamic
vol_name=oem

[userdata-volume]
mode=ubi
image=userdata.ubifs
vol_id=1
vol_size=8MiB
vol_type=dynamic
vol_name=userdata
vol_flags=autoresize
```

mount:

```
ubiattach /dev/ubi_ctrl -m 4 -d 4 -b 5
mount -t ubifs /dev/ubi4_0 /oem
mount -t ubifs /dev/ubi4_1 /uesrdata
```

5.1.3.2 UBIFS Image Making Of Empty Partition

```
umount userdata/
ubidetach -m 4
ubiformat -y /dev/mtd4
ubiattach /dev/ubi_ctrl -m 4 -d 4
ubimkvol /dev/ubi4 -N userdata -m # -N specifies the volume name, - M dynamically adjusts
the partition device autorisize to the maximum
mount -t ubifs /dev/ubi4_0 /userdata
```

5.1.3.3 UBIFS dts Root Mount

```
ubi.mtd=4 root=ubi0:rootfs rootfstype=ubifs
```

5.1.3.4 UBIFS Partition Command Mount

```
ubiattach /dev/ubi_ctrl -m 4 -d 4
```

- -m: mtd num
- -d: ubi binding device
- -b, --max-beb-per1024: maximum expected bad block number per 1024 eraseblock, note that:
    1. 20 in default
    2. Partition image pre production: partition redundancy flash block < --max-beb-per1024 actual value < --max-beb-per1024 set value, that is, the actual value may be smaller than the set value
    3. Command to make empty partition as UBI image: - - max-beb-per1024, the actual value is equal to the set value
    4. The default value of SDK can be set to 10 (this value may not be set in the old version of SDK)
    5. If you need to optimize the space, please set the value flexibly: 4 + the number of blocks occupied by the partition * 1%, for example: flash block size 128KB, OEM space size 16MB, accounting for 128 flash blocks, you can consider filling in the value of 5;

```
mount -t ubifs /dev/ubi4_0 /oem
```

5.1.3.5 UBI Image Partition Overhead

After the UBI image is mounted on the file system, the effective space is less than the partition size. There are mainly UBIFS redundant information and the loss of reserved blocks for bad block replacement.

Accurate calculation

```
UBI overhead = (B + 4) * SP + 0 * (P - B - 4) /* the space cannot be obtained by users */

P - The total number of physical blocks removed on the MTD device
SP - physical erase block size, typically 128KB or 256Kb
SL - logical erase block, i.e. mkfs - e parameter value, usually block_ size - 2 * page_
size
B - F1ash blocks reserved for bad block replacement, related to the ubiattach - b parameter
O - overhead associated with storing EC and vid file headers in bytes, i.e. 0 = SP - sl
```

General case 1

Flash block size 128KB, page size 2KB, 128 MB size, ubiattach - b is reserved by default of 20;

```
SP = block size = 128KB
SL = 128kb - 2 * 2KB = 124KB
B = --max-beb-per1024 * n_ 1024 = 20 * 1 = 20
O = 128KB -124KB = 4KB

UBI overhead = (20 + 4) * 128KB + 4KB * (P - 20 - 4) = 2976KB + 4KB * P
```

If the corresponding partition is 32MB, that is, P = 256, then the final UBI overhead = 2976kb + 4KB * 256 = 4000kb

General case 2

Flash block size 128KB, page size 2KB, 256 MB size, ubiattach - B reserved default 20;

```
SP = block size = 128KB
SL = 128kb - 2 * 2KB = 124KB
B = --max-beb-per1024 * n_ 1024 = 20 * 2 = 40
O = 128KB -124KB = 4KB

UBI overhead = (40 + 4) * 128KB + 4KB * (P - 40 - 4) = 5456KB + 4KB * P
```

If the corresponding partition is 32MB, that is, P = 256, then the final UBI overhead = 5456kb + 4KB * 256 = 6456kb

Detailed reference: flash space overhead chapter http://www.linux-mtd.infradead.org/doc/ubi.html#L_overhead

5.1.4 Support SquashFS In UBI Block

Kernel Configuration

```
+CONFIG_MTD_UBI_BLOCK=y
```

Define Rootfs In dts

```
dts: bootargs: cmdline:

- ubi.mtd=4              : Choose mtd(From 0)
- ubi.block=0,rootfs     : "rootfs" is vol_name(Consult to ubinize.cfg), block=0 is ubi
block index
- root=/dev/ubiblock0_0  : rootfs block dev name, generate from UBI block device drivers
- rootfstype=squashfs    : rootfs type
```

Making SquashFS UBI volume

Buildroot will automatically pack SquashFS image. If need, using mksquashfs command, for example:

```
sudo mksquashfs squashfs-root/ squashfs.img -noappend -always-use-fragments
```

Using ubinize to pack SquashFS image into UBI image:

Firstly generate ubinize.cfg:

```
cat > ubinize.cfg << EOF
[ubifs]
mode=ubi
vol_id=0
vol_type=static
vol_name=rootfs
vol_alignment=1
vol_flags=autoresize
image=/data/rk/projs/rv1126/sdk/buildroot/output/rockchip_rv1126_robot/images/rootfs.squashf
s
EOF
```

Note:

- vol_type: should be static;
- image: input file, path to SquashFS image

then ubinize:

```
ubinize -o rootfs.ubi -p 0x20000 -m 2048 -s 2048 -v ubinize.cfg
```

-p, --peb-size: size of the physical eraseblock of the flash this UBI image is created for in bytes,kilobytes (KiB), or megabytes (MiB) (mandatory parameter)

-m, --min-io-size : minimum input/output unit size of the flash in bytes

-s, --sub-page-size: minimum input/output unit used for UBI headers, e.g. sub-page size in case of NAND flash (equivalent to the minimum input/output unit size by default)

rootfs.ubi is the output file.

Note:

- When using the open source solution in NAND products, Squashfs should not be directly mounted on the mtdblock, because mtdblock does not add bad block detection, so bad block cannot be skipped.

Manually mount UBI block reference

```
ubiattach /dev/ubi_ctrl -m 4 -d 4    /* mount the UBI device */
ubiblock -c /dev/ubi4_0              /* Extending UBI block support on UBI devices */
mount -t squashfs /dev/ubiblock4_0 /oem
```

5.1.5 Optimization Of UBIFS Space Size

As can be seen from the above description, the mirror free space can be optimized through the following three points:

1. Select the appropriate -- max-beb-per1024 parameter, and refer to point 5 of the "- b parameter details" section of "Image making of empty partition"
2. Use UBI multi volume technology to share part of UBIFS redundant overhead. Refer to the description of multi volume production in "Image making"

3. Use the SquashFS supported by UBI block. Refer to the chapter "Support SquashFS In UBI Block"

UBIFS minimum partition:

```
Minimum block num = 4 (fixed reservation) + B + 17 / * B - F1ash blocks reserved for bad
block replacement, related to ubiattach - b parameter*/
```

It can be judged by printing log when ubiattach, for example:

```
ubi4: available PEBs: 7, total reserved PEBs: 24, PEBs reserved for bad PEB handling: 20 /*
B = 20 */
```

If the partition available PEBS + total reserved PEBS < minimum block num, an error will be reported when mounting:

```
mount: mounting /dev/ubi4_ 0 on userdata failed: Invalid argument
```

5.1.6 UBIFS OTA

To upgrade partitions using UBIFS, use the ubiupdatevol tool, the command is as follows:

```
ubiupdatevol /dev/ubi1_0 rootfs.ubifs
```

Note:

- rootfs.ubifs is made by mkfs.ubifs tool

5.1.7 UBIFS FASTMAP

FASTMAP is an experimental and optional feature of UBI (Universal Block Interface) that can be enabled by setting `CONFIG_MTD_UBI_FASTMAP` to 'y'. Once enabled, UBI will evaluate the module parameter "fm_autoconvert". If it is set to 1 (default is 0), UBI will automatically enable fastmap for any attached images. This means UBI will create a new internal volume containing fastmap data for use in fast attach mode in subsequent attachments. Key features include:

- The fastmap pool occupies at most 5% of the physical block space, with at least 2 physical blocks reserved for storing fastmaps.
- It provides certain benefits for larger capacity partitions; actual testing shows no significant benefit for a 128MB partition with UBIFS/UBI Block SquashFS in terms of boot time.

Kernel configuration:

```
CONFIG_MTD_UBI_FASTMAP=y
```

Enable fm_autoconvert after the kernel boots:

```
echo 1 > /sys/module/ubi/parameters/fm_autoconvert
```

After enabling the above configurations, ubiattach device, for example:

```
ubiattach /dev/ubi_ctrl -m 7 -d 7
```

Support disabling the fastmap feature when executing ubiattach. Some older versions of ubiattach tools do not support this feature:

```
ubiattach /dev/ubi_ctrl -m 7 -d 7 -f
```

Since this feature does not provide significant benefits in the general SDK, it is not configured by default. Please enable this property according to actual needs.

5.1.8 Support U-Boot UBIFS

Under uboot, UBIFS only supports read operation, no write/erase operation.

SLC Nand Patches

Refer to RK3308 support, and the patch is as follows:

```
CONFIG_CMD_UBI=y

diff --git a/include/configs/rk3308_common.h b/include/configs/rk3308_common.h
index 1c2b9e4461..bc861acde8 100644
--- a/include/configs/rk3308_common.h
+++ b/include/configs/rk3308_common.h
@@ -57,6 +57,9 @@
 #define CONFIG_USB_FUNCTION_MASS_STORAGE
 #define CONFIG_ROCKUSB_G_DNL_PID        0x330d
+#define MTDIDS_DEFAULT "nand0=rk-nand"
+#define MTDPARTS_DEFAULT "mtdparts=rk-
nand:0x100000@0x400000(uboot),0x100000@0x500000(trust),0x600000@0x600000(boot),0x3000000@0xc
00000(rootfs),0x1200000@0x4000000(oem),0x9f60000@0x6000000(userdata)"
+
 #ifdef CONFIG_ARM64
 #define ENV_MEM_LAYOUT_SETTINGS \
        "scriptaddr=0x00500000\0" \
```

SPI Nand Patches

Refer to RK3568 support, and the patch is as follows:

```
diff --git a/include/configs/rk3568_common.h b/include/configs/rk3568_common.h
index cce44b52a8..eb93455c62 100644
--- a/include/configs/rk3568_common.h
+++ b/include/configs/rk3568_common.h
@@ -92,6 +92,9 @@
        "run distro_bootcmd;"
```

```
 #endif
+#define MTDIDS_DEFAULT "spi-nand0=spi-nand0"
+#define MTDPARTS_DEFAULT "mtdparts=spi-
nand0:0x100000@0x400000(vnvm),0x440000@0x500000(uboot),0x1600000@0xa00000(boot),0x4400000@0x
2000000(rootfs),0x1b60000@0x6400000(userdata)"
+
 /* rockchip ohci host driver */
 #define CONFIG_USB_OHCI_NEW
 #define CONFIG_SYS_USB_OHCI_MAX_ROOT_PORTS     1
(END)
```

menuconfig Enable Configuration

```
#define CONFIG_CMD_UBI = y
```

Mounting

Take rootfs partition as an example, refer to doc for doc/README.ubi.

```
mtdpart
ubi part rootfs
ubifsmount ubi0:rootfs
ubifsls
```

Note:

- As described in the "GPT Partition Extension" section, uboot cmdline passes mtdparts information to the kernel, but MTDPARTS_ DEFAULT definition exception, so kernel cmdline should add mtdparts definition to implement kernel mtd partition definition

5.2 JFFS2 Filesystem

5.2.1 Introduction

The full name of JFFS2 is journaling flash file system version 2. Its function is to manage the journaling file system implemented on MTD devices. Compared with other storage device storage schemes, JFFS2 is not prepared to provide a conversion layer that allows traditional file systems to use such devices. It will only implement the file system with log structure directly on the MTD device. JFFS2 will scan the log contents of MTD device during installation and re-establish the file system structure itself in RAM.

5.2.2 Notice

1. The mkfs.jffs2 tool cannot prefabricate a 4KB erase size JFFS2 image.
    1. The default mtd-utils tool does not support 4KB erasing.
    2. The RK buildroot tool modifies mtd-utils to support 4KB erasing, refer to the modification:

```
```

```
diff --git a/jffsX-utils/mkfs.jffs2.c b/jffsX-utils/mkfs.jffs2.c
index 9aa6c39..3cec529 100644
--- a/jffsX-utils/mkfs.jffs2.c
+++ b/jffsX-utils/mkfs.jffs2.c
@@ -1668,6 +1668,11 @@ int main(int argc, char **argv)

                        /* If it's less than 8KiB, they're not allowed */
                        if (erase_block_size < 0x2000) {
+                           if (erase_block_size == 0x1000) {
+                               warnmsg("Set Erase size to 4KB with Experimental.\n");
+                               erase_block_size = 0x1000;
+                               break;
+                           }
                            fprintf(stderr, "Erase size 0x%x too small. Increasing to 8KiB
minimum\n",
                                    erase_block_size);
                            erase_block_size = 0x2000;
diff --git a/jffsX-utils/sumtool.c b/jffsX-utils/sumtool.c
index 68268a9..836b426 100644
--- a/jffsX-utils/sumtool.c
+++ b/jffsX-utils/sumtool.c
@@ -194,6 +194,11 @@ static void process_options (int argc, char **argv)

                        /* If it's less than 8KiB, they're not allowed */
                        if (erase_block_size < 0x2000) {
+                           if (erase_block_size == 0x1000) {
+                               warnmsg("Set Erase size to 4KB with Experimental.\n");
+                               erase_block_size = 0x1000;
+                               break;
+                           }
                            warnmsg("Erase size 0x%x too small. Increasing to 8KiB
minimum\n",
                                  erase_block_size);
                            erase_block_size = 0x2000;
--
2.23.0-rc1
```

2. The kernel JFFS2 supports 4KB erase settings for NOR flash and supports the flash_erase -j command for low-level formatting to generate JFFS2 4KB partitions.

5.2.3 Configuration

Kernel Configuration:

```
CONFIG_JFFS2_FS=y
CONFIG_MTD_SPI_NOR_USE_4K_SECTORS=n /* Pre-production image scheme only supports 64KB erase
alignment */
```

Note:

- For kernel versions 5.10 and higher, the RK SPI NOR storage driver uses the standard MTD NOR flash driver framework, and most of the chip code supports the `SECT_4K` attribute. It supports switching between 64KB and 4KB different erase granularity through the macro `CONFIG_MTD_SPI_NOR_USE_4K_SECTORS`. The main differences between different erase granularities are as follows:
  - The 64KB erase speed is faster, with a typical time of 0.12s/0.15s for the GD25Q256E, but the 64KB alignment results in lower space utilization.
  - The 4KB erase speed is relatively slower, with a typical time of 30ms for the GD25Q256E, which is less than one-third the efficiency of block erasing, but the 4KB alignment results in higher space utilization.

5.2.4 JFFS2 Image Making

For example:

```
mkfs.jffs2 -r data/ -o data.jffs2 -e 0x10000 --pad=0x400000 -s 0x1000 -n
Options:
 --pad [=SIZE]            Pad output to SIZE bytes with 0xFF. If SIZE is
                          not specified, the output is padded to the end of
                          the final erase block
  -r, -d, --root=DIR      Build file system from directory DIR (default: cwd)
  -s, --pagesize=SIZE     Use page size (max data node size) SIZE.
                          Set according to target system's memory management
                          page size (default: 4KiB)
  -e, --eraseblock=SIZE   Use erase block size SIZE (default: 64KiB)
  -n, --no-cleanmarkers   Don't add a cleanmarker to every eraseblock
```

Note:

- --pad: format with partition size
- -e: erase size: The default only supports 64KB, and it is possible to support a 4KB scheme by modifying mtd-utils. For details, see the "Notices" section.
- -s: 4KB in default

6. Flash Programmer

6.1 SPI Nand Flash Programmer - GPT Partition Extension

6.1.1 Make SPI Nand Images

Input Files: SDK Output Files For PC Tools

```
[/IMAGES] tree
.
├── parameter.txt
├── MiniLoaderAll.bin
├── uboot.img
├── boot.img
├── rootfs.img
├── oem.img
└── update.img                  // For make programmer imagers
```

Make Images

tool programmer_image_tool is in SDK rkbin/tools/, command:

```
./tools/programmer_image_tool  --help
NAME
        programmer_image_tool - creating image for programming on flash
SYNOPSIS
        programmer_image_tool [-iotbpsvh]
DESCRIPTION
        This tool aims to convert firmware into image for programming
        From now on,it can support slc nand(rk)|spi nand|nor|emmc.
OPTIONS:
          -i    input firmware
          -o    output directory
          -t    storage type,range in[SLC|SPINAND|SPINOR|EMMC]
          -b    block size,unit KB
          -p    page size,unit KB
          -s    oob size,unit B
          -2    2k data in one page
          -l    using page linked l
```

Assume that: rv1126 block size 128KB page size 2KB flash:

```
./tools/programmer_image_tool  -i update.img -b 128 -p 2 -t spinand -o out
input firmware is 'update.img'
block size is '128'
page size is '2'
flash type is 'spinand'
output directory is 'out'
writing idblock...
start to write partitions...gpt=1
preparing gpt saving at out/gpt.img
writing gpt...OK
preparing trust saving at out/trust.img
writing trust...OK
preparing uboot saving at out/uboot.img
writing uboot...OK
preparing boot saving at out/boot.img
writing boot...OK
preparing rootfs saving at out/rootfs.img
writing rootfs...OK
```

```
preparing recovery saving at out/recovery.img
writing recovery...OK
preparing oem saving at out/oem.img
writing oem...OK
preparing userdata:grow saving at out/userdata.img
writing userdata:grow...OK
preparing misc saving at out/misc.img
writing misc...OK
creating programming image ok.
```

Note:

- 4KB page size programmer_image_tool add -2 parameter

output files: Using For Flash Programmer

```
out
├── boot.img
├── gpt.img
├── idblock.img            //Making multiple backup images of idblock_mutli_copies.img on
demand
├── misc.img
├── oem.img
├── recovery.img
├── rootfs.img
├── trust.img
├── uboot.img
└── userdata.img
```

Note:

- IDB multi backup command reference, usually double backup can be:

```
cat out/idblock.img > out/idblock_mutli_copies.img   // 1 copy
cat out/idblock.img >> out/idblock_mutli_copies.img  // 2 copies
```

6.1.2 SPI Nand Flash Programmer OperationProgrammer Address

Assume that flash block size is 128KB, AndroidTools and it's corresponding flash programmer setting could be like this:

Input File: SDK output	AndroidTools Start(sector)	Flash Programmer Images	Programmer Start(block)	End(block)	Size(block)	Note
paramter.txt	0	gpt.img	0x0	0x1	0x1	Note 1
MiniLoaderAll.bin	0	idblock_mutli_copies.img	0x1	0x7	0x6	Note 2
uboot.img	0x2000	uboot.img	0x20	0x47	0x20	**Note 3**
boot.img	0x4800	boot.img	0x48	0xa0	0x50	
...	...	...	...			
xxx.img	0x3E000	xxx.img	0x3e0	0x3fb	0x18	Note 4

Table Note:

1. gpt.img should be placed in block 0;
2. idblock_mutli_copies.img should be placed from block 1 to block 7, the image size is limit to 7 blocks;
3. Except gpt.img and idblocks.img, other images should be place in the address based on parameter.txt address, 512B/sector, flash programmer Start block = sectors * 512B / block_size:
   - 128KB block size: sectors / 0x100
   - 256KB block size: sectors / 0x200
   - Except gpt.img, other images size should less then partition size from 1 to 2 block size to make bad block replacement possible;
4. Resert the last 4 flash block size for bad block table, consider defining the reverted partition to avoid user use or future misuse.

Other Note

1. Because the RK spinand controller FSPI (formerly known as SFC) does not integrate the ECC module, it needs to rely on the ECC function of the device itself. Therefore, the image used by the programmer does not contain oob data, the oob space is filled by the programmer itself, and the ECC function of the spinand device is enabled in the tool interface of the programmer;
2. Erase all good blocks for none empty flash;
3. Enable verification.

6.2 SLC Nand Flash Programmer - GPT Partition Extension

6.2.1 Make SLC Nand Images

Input Files: SDK Output Files For PC Tools

```
[/IMAGES] tree
.
├── parameter.txt
├── MiniLoaderAll.bin
├── uboot.img
├── boot.img
├── rootfs.img
├── oem.img
└── update.img                  // For make programmer imagers
```

Make Images

tool programmer_image_tool is in SDK rkbin/tools/, command:

```
./tools/programmer_image_tool  --help
NAME
        programmer_image_tool - creating image for programming on flash
SYNOPSIS
        programmer_image_tool [-iotbpsvh]
DESCRIPTION
        This tool aims to convert firmware into image for programming
        From now on,it can support slc nand(rk)|spi nand|nor|emmc.
OPTIONS:
          -i    input firmware
          -o    output directory
          -t    storage type,range in[SLC|SPINAND|SPINOR|EMMC]
          -b    block size,unit KB
          -p    page size,unit KB
          -s    oob size,unit B
          -2    2k data in one page
          -l    using page linked l
```

Assume that: rv1126 block size 128KB page size 2KB oob size 128 flash:

```
./tools/programmer_image_tool -i update.img -b 128 -p 2 -s 128 -t slc -o out
input firmware is 'update.img'
block size is '128'
page size is '2'
oob size is '128'
flash type is 'slc'
2k data page on.
output directory is 'out'
writing idblock...
start to write partitions...gpt=1
preparing gpt saving at out/gpt.img
writing gpt...OK
preparing trust saving at out/trust.img
writing trust...OK
preparing uboot saving at out/uboot.img
writing uboot...OK
preparing boot saving at out/boot.img
writing boot...OK
```

```
preparing rootfs saving at out/rootfs.img
writing rootfs...OK
preparing recovery saving at out/recovery.img
writing recovery...OK
preparing oem saving at out/oem.img
writing oem...OK
preparing userdata:grow saving at out/userdata:grow.img
writing userdata:grow...OK
preparing misc saving at out/misc.img
writing misc...OK
creating programming image ok.
```

Note:

- Add '-l' tag for RK3326/PX30/RK3568
- Using 4KB page size flash in RV1126/RK3326/PX30/RK3308, add -2 to programmer_image_tool
- The output image is aligned with "block size with oob", check the detail from "Nand Flash Information" chapter

output files: Using For Flash Programmer

```
out
├── boot.img
├── gpt.img
├── idblock.img            //Making multiple backup images of idblock_mutli_copies.img on
demand
├── misc.img
├── oem.img
├── recovery.img
├── rootfs.img
├── trust.img
├── uboot.img
└── userdata.img
```

Note:

- IDB multi backup command reference, usually double backup can be:

```
cat out/idblock.img > out/idblock_mutli_copies.img   // 1 copy
cat out/idblock.img >> out/idblock_mutli_copies.img  // 2 copies
```

6.2.2 SLC Nand Flash Programmer Operation

Programmer Address

Assume that flash block size is 128KB, AndroidTools and it's corresponding flash programmer setting could be like this:

Input File: SDK output	AndroidTools Start(sector)	Flash Programmer Images	Programmer Start(block)	End(block)	Size(block)	Note
paramter.txt	0	gpt.img	0x0	0x1	0x1	Note 1
MiniLoaderAll.bin	0	idblock_mutli_copies.img	0x1	0x7	0x6	Note 2
uboot.img	0x2000	uboot.img	0x20	0x47	0x20	**Note 3**
boot.img	0x4800	boot.img	0x48	0xa0	0x50	
...	...	...	...			
xxx.img	0x3E000	xxx.img	0x3e0	0x3fb	0x18	Note 4

Table Note:

1. gpt.img should be placed in block 0;
2. idblock_mutli_copies.img should be placed from block 1 to block 7, the image size is limit to 7 blocks;
3. Except gpt.img and idblocks.img, other images should be place in the address based on parameter.txt address, 512B/sector, flash programmer Start block = sectors * 512B / block_size:
   - 128KB block size: sectors / 0x100
   - 256KB block size: sectors / 0x200
   - Except gpt.img, other images size should less then partition size from 1 to 2 block size to make bad block replacement possible;
4. Resert the last 4 flash block size for bad block table, consider defining the reverted partition to avoid user use or future misuse.

Other Note

1. The image contain OOB data;
2. Erase all good blocks for none empty flash;
3. Enable verification

6.3 SPI Nor Flash Programmer - GPT Partition Extension

6.3.1 Make SPI Nor Images

Input Files: SDK Output Files For PC Tools

```
[/IMAGES] tree
.
├── parameter.txt
├── MiniLoaderAll.bin
├── uboot.img
├── boot.img
├── rootfs.img
├── oem.img
└── update.img                  // For make programmer imagers
```

Make Images

tool programmer_image_tool is in SDK rkbin/tools/, command:

```
./tools/programmer_image_tool  --help
NAME
        programmer_image_tool - creating image for programming on flash
SYNOPSIS
        programmer_image_tool [-iotbpsvh]
DESCRIPTION
        This tool aims to convert firmware into image for programming
        From now on,it can support slc nand(rk)|spi nand|nor|emmc.
OPTIONS:
          -i    input firmware
          -o    output directory
          -t    storage type,range in[SLC|SPINAND|SPINOR|EMMC]
          -b    block size,unit KB
          -p    page size,unit KB
          -s    oob size,unit B
          -2    2k data in one page
          -l    using page linked l
```

For example:

```
./tools/programmer_image_tool -i update.img  -t SPINOR -o ./out
input firmware is 'update.img'
flash type is 'SPINOR'
output directory is './out'
writing idblock...
start to write partitions...gpt=1
preparing gpt at 0x00000000
writing gpt...OK
preparing trust at 0x00002800
writing trust...OK
preparing uboot at 0x00002000
writing uboot...OK
preparing boot at 0x00009800
writing boot...OK
preparing rootfs at 0x0000c800
writing rootfs...OK
preparing recovery at 0x00003800
writing recovery...OK
```

```
preparing misc at 0x00003000
writing misc...OK
creating programming image ok.
```

output files: Using For Flash Programmer

```
tree
.
└── out_image.img
```

6.3.2 SPI Nor Flash Programmer Operation

The image output from programmer_image_tool is burned to SPI Nor 0 address.

6.4 SPI Nand Flash Programmer - ENV Partition Extension

6.4.1 Make SPI Nand Images

Input Files: SDK Output Files

```
[/IMAGES] tree
.
├── boot.img
├── download.bin
├── env.img
├── idblock.img
├── oem_2KB_128KB_48MB.ubi
├── oem_2KB_256KB_48MB.ubi
├── oem_4KB_256KB_48MB.ubi
├── oem.img
├── rootfs_2KB_128KB_32MB.ubi
├── rootfs_2KB_256KB_32MB.ubi
├── rootfs_4KB_256KB_32MB.ubi
├── rootfs.img
├── sd_update.txt
├── tftp_update.txt
├── uboot.img
├── update.img
├── userdata_2KB_128KB_32MB.ubi
├── userdata_2KB_256KB_32MB.ubi
├── userdata_4KB_256KB_32MB.ubi
└── userdata.img
```

- The SDK output image can be used for both PC tools programing and programmer loading.
- The default idblock.img image supports 2KB page size spinand, but for 4KB page size spinand, it needs to be re-created. The output file is still idblock.img, and the command to create it is:

```
./rkbin/tools/programmer_image_tool  -i download.bin -b 256 -p 4 -2 -t spinand -o ./
```

6.4.2 SPI Nand Flash Programmer Operation Programmer Address

Assume that flash block size is 128KB, AndroidTools and it's corresponding flash programmer setting could be like this:

Input File: SDK output	AndroidTools Start(sector)	Flash Programmer Images	Programmer Start(block)	End(block)	Size(block)	Note
env.img	0	env.img	0x0	0x1	0x1	Note 1
idblock.img	0x40000	idblock_mutli_copies.img	0x1	0x3	0x6	Note 2
uboot.img	0x80000	uboot.img	0x4	0x7	0x4	**Note 3**
boot.img	0xC0000	boot.img	0x8	0x47	0x40	
...	...	...	...			
xxx.img	0x3E000	xxx.img	0x2c6	0x3fc	0x136	Note 4

Table Note:

1. env.img should be placed in block 0;
2. idblock_mutli_copies.img should be placed from block 1 to block 7, the image size is limit to 7 blocks;
3. Except evb.img and idblocks.img, other images should be place in the address based on env.img address. Unit: byte, flash programmer Start block = address / block_size:
    - 128KB block size: address / 0x20000
    - 256KB block size: address / 0x40000
    - Except env.img, other images size should less then partition size from 1 to 2 block size to make bad block replacement possible;
4. Resert the last 4 flash block size for bad block table, consider defining the reverted partition to avoid user use or future misuse.

Other Note

1. Because the RK spinand controller FSPI (formerly known as SFC) does not integrate the ECC module, it needs to rely on the ECC function of the device itself. Therefore, the image used by the programmer does not contain oob data, the oob space is filled by the programmer itself, and the ECC function of the spinand device is enabled in the tool interface of the programmer;
2. Erase all good blocks for none empty flash;
3. Enable verification.

6.5 SPI Nor Programmer - ENV Partition Expansion

6.5.1 Make SPI Nor Images

Input Files: Images generated by the SDK for PC tool burning

```
[/IMAGES] tree
.
├── env.img
├── idblock.img
├── uboot.img
├── boot.img
├── rootfs.img
├── oem.img
└── update.img                  // Used to create the burning image
```

Creating the Image

For example:

```
./tools/programmer_image_tool -i update.img  -t SPINOR -o ./out
input firmware is 'update.img'
flash type is 'SPINOR'
output directory is './out'
start to write partitions...env
preparing env at 0x00000000
writing env...OK
preparing idblock at 0x00000200
writing idblock...OK
preparing uboot at 0x00000400
writing uboot...OK
preparing boot at 0x00000600
writing boot...OK
preparing rootfs at 0x00004600
writing rootfs...OK
preparing oem at 0x00014600
writing oem...OK
preparing userdata at 0x0002c600
writing userdata...OK
creating programming image ok.
```

Output File: Image for Burning with the Burner

```
tree
.
└── out_image.img
```

6.5.2 SPI Nand Flash Programmer Operation Programmer Address

Burn the image generated by the `programmer_image_tool` to address 0 of the SPI Nor memory.

7. Test items

7.1 Nand Flash Product Testing Project

Applicable to SLC PP Nand and SPI Nand products.

7.1.1 flash_stress_test Read/Write Stress Test

Test Objective To test the stability of SPI Nand under read/write stress testing. The relevant tests are usually integrated into the SDK buildroot, simply by adding the rockchip_test option.

Test Devices 10 target devices with approximately 30MB of reserved space in the userdata partition.

Test Method

1. Select the "3 flash_stress_test" option in `./rockchip_test/rockchip_test.sh` in the SDK buildroot, ensuring that the device remains powered on during the test.
2. Record the logs during the testing process.
3. Stop the test by pressing `ctrl + c` on the host machine.
4. Judge the test results: If the device continues to function normally after 24 hours, the test is considered passed; otherwise, it fails. It is recommended to search for keywords such as "ubi" and "error" to check for any obvious error messages.

7.1.2 power_lost_test Abnormal Power Loss Test

Test Objective Nand products often encounter abnormal power loss situations during operation, especially for products without batteries. It is necessary to perform targeted robustness testing.

The relevant tests are usually integrated into the SDK buildroot, simply by adding the rockchip_test option.

Test Devices 10 target devices with approximately 30MB of reserved space in the userdata partition.

Test Method

1. Select the "18 nand power lost test" option in `./rockchip_test/rockchip_test.sh` in the SDK buildroot. After the device is re-powered, it will enter continuous testing mode, and record the serial port logs.
2. Set the power-off device to supply power for 14 seconds and then turn off for 3 seconds, i.e., every 17 seconds per group, for a 24-hour test, resulting in approximately 5000 instances of abnormal power loss. Ensure that the device completes the power-on process and runs the test script within 17 seconds after each power loss.

3. Record the logs during the testing process.
4. Stop the test by entering the following command on the host machine: `echo off > /data/cfg/rockchip_test/reboot_cnt`.
5. Judge the test results: If the device continues to function normally after 24 hours, the test is considered passed; otherwise, it fails. It is recommended to search for keywords such as "ubi" and "error" to check for any obvious error messages.

8. FAQ

8.1 Nand Flash Information

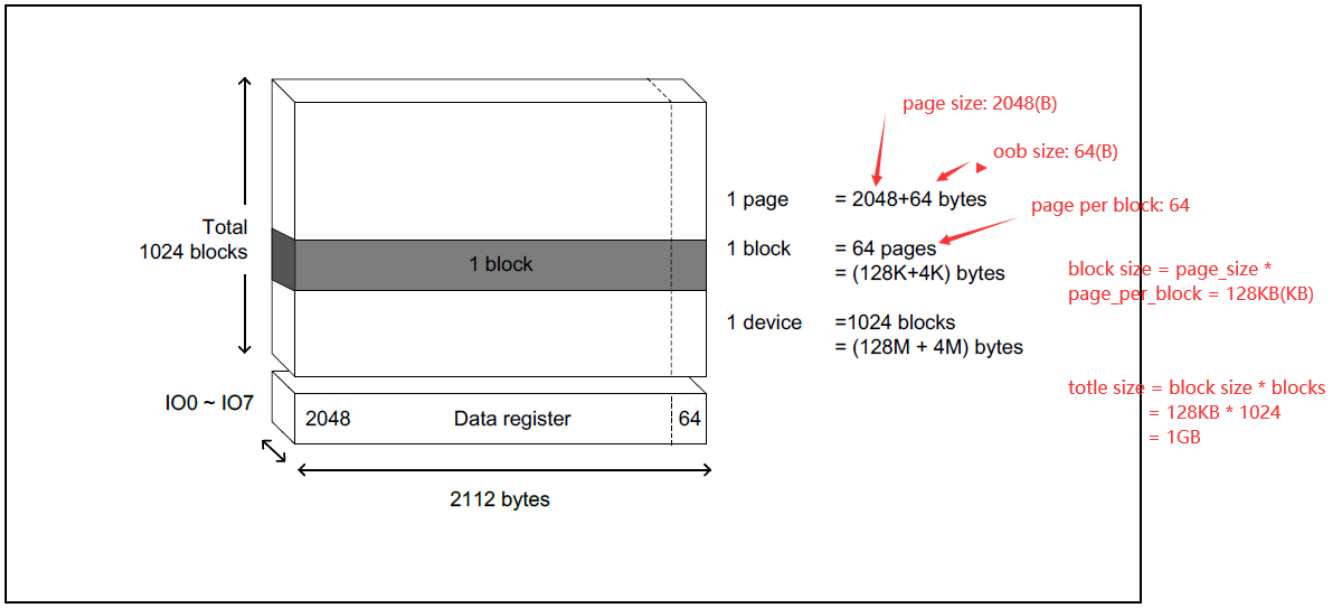
Making UBIFS images, IDB images(Pre loader) should depend on the concrete Nand flash information, including follow information:

- page size, SLC Nand is usually in 2KB or 4KB;
- oob size, SLC Nand is usually in 64B, 128B or 256B.
- page per block, SLC Nand is usually in 64 or 128;
- block size = page_size * page_per_block, SLC Nand is usually 128KB or 256KB;
- block size with oob = (page_size + oob_size) * page_per_block;

The default configuration is mostly base on 2KB page size and 128 block size flash, change it if need when you perform the following process:

- Making Programmer images
- Making UBIFS filesystem images

The Nand flash information is mostly like this:



8.2 IDB Layout

IDB is the packaging firmware of ddr.bin and spl.bin, and the first level firmware after maskrom. Its layout is as follows:

- The upgrade of PC tools is stored in flash block 1 ~ 7 and filled with multiple backups;
- Suggestion of programmer image layout: refer to the description of "Programmer address" in "Flash Programmer" chapter.

8.3 SPI Flash Kernel speed calculation

mtd_debug Test Commands

Choose an unused mtd partition for testing, and ensure that the partition is not mounted with a file system to avoid interference during the test. Use the mtd_debug test command for speed testing:

```
# Test Preparation
dd if=/dev/random of=/tmp/test4M bs=1M count=4
mtd_debug read /dev/mtd6 0 0x400000 /tmp/test_backup
sync

# Write Speed Test
mtd_debug erase /dev/mtd6 0 0x400000 && sync
time mtd_debug write /dev/mtd6 0 0x400000 /tmp/test4M && time sync

# Read Test
time mtd_debug read /dev/mtd6 0 0x400000 /tmp/test4M && time sync

# Erase Test
time mtd_debug erase /dev/mtd6 0 0x400000 && time sync

# Test End
time mtd_debug write /dev/mtd6 0 0x400000 /tmp/test_backup && time sync
```

SPI Nand Flash Chip Rate Calculation

Take a W25N01KVxxIR SPI Nand Flash chip as an example, and through reading the manual "AC Electrical Characteristics" chapter, get the following information:

- Block size: 128KB, Page size: 2KB
- Erase time: tBE typical 2ms
- Program time: tPP typical 250us
- Read time (ECC enable): tRD typical 45us

Perform theoretical calculations using typical values:

- Programming rate needs to be included in io rate, assuming io 100MHz single line, read tIO 160us, programming rate page_size / (tPP + tIO), about 5MB/s The kernel default only supports single line programming

- Reading rate needs to be included in io rate, assuming io 100MHz quad line, read tIO 40us, reading rate page_size / (tRD + tIO) , about 23MB/s
- Erase rate does not need to be counted in io delay, so the erase rate block_size / tBE, about 64MB/s

SPI Nor Flash Chip Rate Calculation

Take an XT25F128B SPI Nor Flash chip as an example, and through reading the manual "AC Electrical Characteristics" chapter, get the following information:

- Block size: 64KB, Page size: 256B
- Erase time: tBE typical 200ms
- Program time: tPP typical 300us
- Read time (ECC enable): close to IO rate, tIO

Perform theoretical calculations using typical values:

- Programming rate needs to be included in io rate, assuming io 100MHz single line, read tIO 160us, programming rate page_size / (tPP + tIO), about 557KB/s The kernel default only supports single line programming
- Reading rate is only io rate, assuming io 100MHz quad line, reading rate is about 50MB/s
- Erase rate does not need to be counted in io delay, so the erase rate block_size / tBE, about 320KB/s

8.4 Nand Product Lifespan

The nominal write life cycle (P/E cycles) of an SPI Nand flash is commonly referred to as the number of P/E cycles. The manufacturer's specifications usually specify a value of 100K for this parameter.

Predicting the Effect of Redundancy on the Flash Product Lifespan

The original manufacturer's specifications often have constraints on how the flash can be used, such as:

- Avoid frequent writes to the same block of data, as this can affect the flash's lifespan
- Some types of flash also require some level of data balance, otherwise it can affect the charge balance

Therefore, it may be beneficial to add redundancy and take into account factors such as the actual flash product lifespan, differences between different manufacturers, and the impact of file systems on the flash's performance. For example, you could estimate that the flash will last for an additional 50K cycles by accounting for 50% of the total write operations.

Calculating the Maximum Data Write Capacity of the Flash Product

Assuming a partition size of 100M, the flash can support a maximum of 50K x 100M write operations after accounting for redundancy. However, this is only the write capacity of the flash without considering any file system strategies. For UBIFS, the file system typically uses redundancy around 20%, which means that there will be about 128KB of flash block space dedicated to UBIFS algorithm information. Therefore, the effective data write capacity would be 4T, assuming a compression rate of 50%. If you consider the compression file system supported by UBIFS, the effective data write capacity would be even higher. The above calculations are based on the main factors influencing the flash's performance and should be taken with a grain of salt. The actual results may vary depending on the file system used, the compression scheme used by the file system, and the actual lifespan of the flash product.

Actual Testing of the Flash Product Lifespan - UBIFS File System

When the device is started, the partition P/E cycles information is printed:

```
ubi0: max/mean erase counter: 164/10, WL threshold: 4096, image sequence number: 207598880
```

This indicates:

- The maximum value for the number of data blocks that can be erased within a partition is shown here
- After a certain period of time, confirm the change in the maximum value, which can then be used to estimate the flash product's lifespan, for example, if the maximum value increases by 100 times after 24 hours of operation, then for an SPI Nand with 50K cycles, the estimated lifespan would be 500 days.

Actual Testing of the Flash Product Lifespan - Without a File System

Test Patch, Monitoring the Userdata Partition as an Example:

```
diff --git a/drivers/mtd/mtdcore.c b/drivers/mtd/mtdcore.c
index ad527bdbd463..ae34665e5c89 100644
--- a/drivers/mtd/mtdcore.c
+++ b/drivers/mtd/mtdcore.c
@@ -36,6 +36,28 @@
 struct backing_dev_info *mtd_bdi;

+#include <linux/moduleparam.h>
+
+static int e_count;
+module_param(e_count, int, S_IRUGO);
+MODULE_PARM_DESC(e_count, "e_count");
+
+static int p_count;
+module_param(p_count, int, S_IRUGO);
+MODULE_PARM_DESC(p_count, "p_count");
+
+static int r_count;
+module_param(r_count, int, S_IRUGO);
+MODULE_PARM_DESC(r_count, "r_count");
+
+static inline int mtd_trace(struct mtd_info *mtd, char *cmp_name, int len)
+{
+       if (cmp_name && strcmp(mtd->name, cmp_name))
+               return 0;
+       else
+               return len;
+}
+
 #ifdef CONFIG_PM_SLEEP

 static int mtd_cls_suspend(struct device *dev)
@@ -1336,6 +1358,8 @@ int mtd_erase(struct mtd_info *mtd, struct erase_info *instr)
        struct erase_info adjinstr;
        int ret;
```

```
+       e_count += mtd_trace(mtd, "userdata", mtd->erasesize);
+
        instr->fail_addr = MTD_FAIL_ADDR_UNKNOWN;
        adjinstr = *instr;

@@ -1467,6 +1491,8 @@ int mtd_read(struct mtd_info *mtd, loff_t from, size_t len, size_t
*retlen,
        };
        int ret;

+       r_count += mtd_trace(mtd, "userdata", len);
+
        ret = mtd_read_oob(mtd, from, &ops);
        *retlen = ops.retlen;

@@ -1483,6 +1509,8 @@ int mtd_write(struct mtd_info *mtd, loff_t to, size_t len, size_t
*retlen,
        };
        int ret;

+       p_count += mtd_trace(mtd, "userdata", len);
+
        ret = mtd_write_oob(mtd, to, &ops);
        *retlen = ops.retlen;

(END)
```

Operation Method:

- Update the firmware with the patch applied, and after a period of testing, enter the following commands to obtain the e/p/r (erase/program/read) data volume, in Bytes:

```
cat /sys/module/mtd/parameters/e_count
cat /sys/module/mtd/parameters/p_count
cat /sys/module/mtd/parameters/r_count
```

Calculate Lifespan:

- Calculate the hourly written data volume, in MB/hour.
- According to the spec P/E cycle of the NAND, for example, 100K (common), if the partition is 50MB, the nominal data volume is 5T. Therefore, the product lifespan is estimated as "Total data volume / Hourly written data volume."

Other Notes:

- The P/E lifespan of the flash is the main indicator of the flash lifespan, and other indicators have a relatively lower impact.

Other Notes:

- The flash P/E cycle lifespan is a key indicator of the flash's overall performance, while other metrics may have a smaller impact.

8.5 Nand Power-off Exception Issues

Key Indicators of Power-off

Abnormal power-off of devices mainly refers to power-down behaviors that do not go through the standard de-initialization process of the system, which may cause the Nand chip to work under unstable voltage levels.

The main impact is on the erasing and programming actions of Nand flash. Typically, the erasing time reaches several milliseconds, and the programming time reaches hundreds of microseconds. The working voltage of 3V3 chips is usually at 2.7V or above, with the actual values based on the manual.

Typical Exceptions

Abnormal power-off of flash mainly leads to the following two situations:

- Incomplete data being written
- During the power-off process, if the flash is processing commands under unstable voltage, it may cause random exceptions

Protection Mechanisms

Incomplete data being written:

- The file system has a power-off recovery mechanism to ensure that the file system is not affected by incomplete data, which may result in data loss.

Unstable low voltage causing random exceptions:

- PMIC Solution: The PMIC detects the voltage of one route, and when the voltage level drops below a certain value, it triggers a CPU reset signal. The host no longer initiates Nand erase/write commands, and the flash peripheral circuit enters a discharge process, maintaining a working voltage for about 1ms to ensure the last data operation of Nand is normal.
- Reset IC Solution: The principle is the same as the PMIC solution. When the detected voltage drops below the threshold voltage, it triggers a CPU reset signal.
- RK NPOR Solution: The principle is similar to the PMIC solution. When the detected voltage drops below the threshold voltage, it triggers an interrupt signal and supports querying the voltage status.

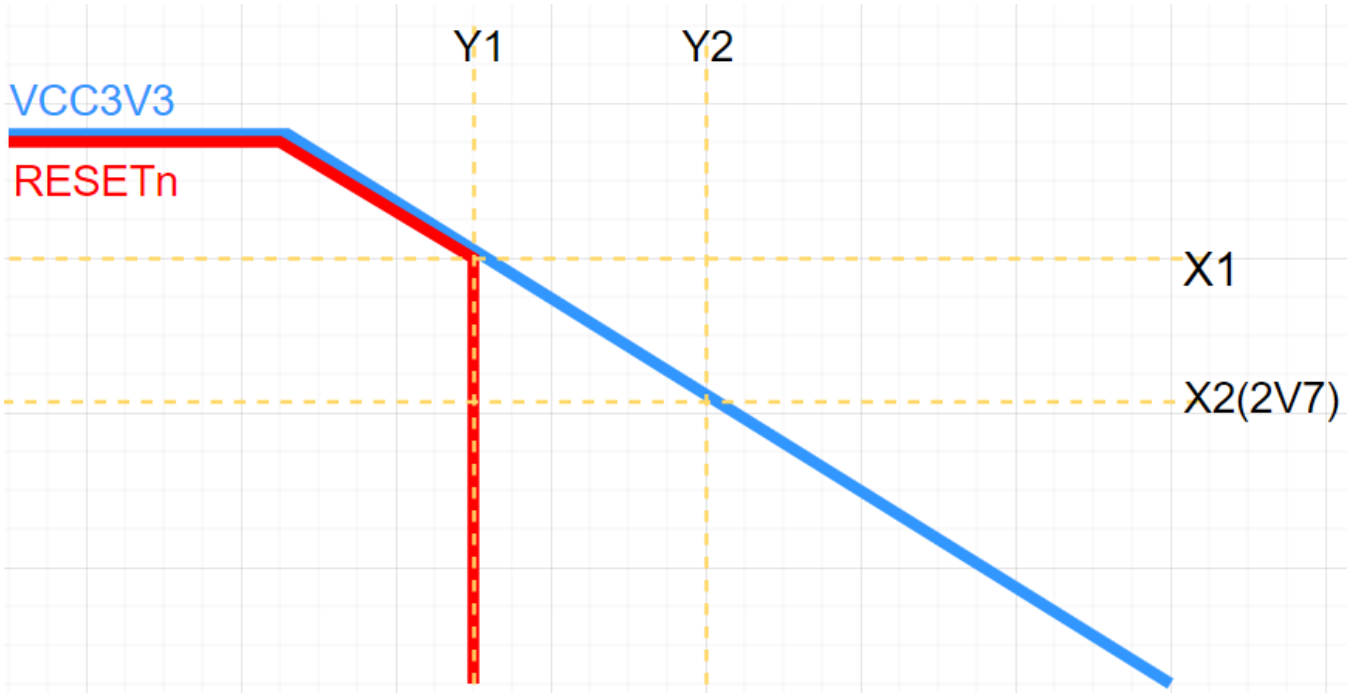
How to Avoid Abnormal Power-off Issues

- It is recommended that Nand products be designed with a battery or a constant power supply to avoid abnormal power-off behaviors.
- If abnormal power-off behaviors cannot be avoided, it is recommended to optimize the hardware design to protect the working voltage of Nand during power-off:
    - PMIC/Reset IC/RK NPOR solution detects VCC IO voltage, monitors power-off behavior, triggers CPU reset to avoid initiating Nand erase/write commands under low voltage.
    - Further optimize the selection of VCC3V3 capacitors based on the drawings provided by RK to maintain the working voltage of the last flash erase/write operation during Nand power-off, usually for tPROG (page programming time). If possible, further optimization to tBERS (block erase time) is recommended.
    - Reduce power-off behaviors during flash read/write processes.

- Implement backup and recovery mechanisms for key data:
  - Firmware dual backup
  - Key data stored in a read-only file system
  - If a power-off exception occurs in a writable partition, it is recommended to have a recovery mechanism to ensure the device can operate normally or implement OTA.

How to Determine if the Abnormal Power-off Timing is Reasonable

In the actual product and business environment, set up a "power_lost_test" test environment, and then capture the timing of the VCC3V3/RESETn signals during power-off:



VCC3V3/RESETn signal timing measurement requirements:

- The oscilloscope interface provides the following information:
  - RESETn trigger point Y1 axis
  - VCC3V3 power-off to 2V7 Y2 axis, based on the minimum voltage level of the SPI Nand manual
  - Confirm the ΔY1Y2 axis time
- Measure multiple times, select the minimum ΔY1Y2 value, and consider signal superposition processing.
- In the SOC solutions implemented with RKNPOR, some schemes internally encapsulate the RESETn signal, making it unmeasurable. Therefore, when measuring the power-down timing, the last falling edge of the FSPI CSN is used as a substitute for the RESETn signal, such as in the case of RV1106.

8.6 Flash Kernel Signal Test Script

The open source MTD framework has a mature MTD_TESTS framework, located at the source path drivers/mtd/tests.

Spinand Device Patches

To avoid excessive read status bit behavior during signal writing tests, add delays to skip polling time.

```
diff --git a/drivers/mtd/nand/spi/core.c b/drivers/mtd/nand/spi/core.c
index f0ba92a09ae5..3f1d09200f74 100644
--- a/drivers/mtd/nand/spi/core.c
+++ b/drivers/mtd/nand/spi/core.c
@@ -573,6 +573,7 @@ static int spinand_write_page(struct spinand_device *spinand,
        if (ret)
                return ret;

+       mdelay(1);
        ret = spinand_wait(spinand, &status);
        if (!ret && (status & STATUS_PROG_FAILED))
                ret = -EIO;
```

Macro Switches

CONFIG_MTD_TESTS

Output ko Files

mtd_readtest.ko, mtd_torturetest.ko

Push Read/Write Pressure Test ko Files

```
adb push mtd_readtest.ko mtd_torturetest.ko /data
```

Read/Write Commands

```
insmod /data/mtd_readtest.ko dev=0 cycles_count=10                  # Read test, you can
modify cycles_count to adjust the test duration
rmmod mtd_readtest                                                  # Read test clear, if
you want to re-test, re-insmod

insmod /data/mtd_torturetest.ko dev=0 check=0 cycles_count=10 random_pattern=1      # Write
test, you can modify cycles_count to adjust the test duration
rmmod mtd_torturetest                                               # Write test clear, if
you want to re-test, re-insmod
```

Note:

- For read testing, it is recommended to choose a partition with large capacity and valid image for testing, so that the data line is easy to show high and low level changes.
- For write testing, it is recommended to choose an idle partition to avoid affecting the system operation due to writing data.

8.7 Flash has aprobable startup abnormality or read file system verification error

It is recommended to reduce the frequency of testing. If the machine test is normal, it is suggested to measure the signal and make further optimization and adjustments for flash signal quality.

mtd single line flash patch:

Modify the spi-rx-bus-width of the dts spiflash device node to 1, which means single-line write and single-line read.

rkflash single line flash patch:

```
diff --git a/drivers/rkflash/sfc_nand.c b/drivers/rkflash/sfc_nand.c
index 4accf3d791c6..98b1cfecc821 100644
--- a/drivers/rkflash/sfc_nand.c
+++ b/drivers/rkflash/sfc_nand.c
@@ -1179,6 +1179,8 @@ u32 sfc_nand_init(void)
                return -ENOMEM;
        }

+       p_nand_info->feature &= ~(FEA_4BIT_READ | FEA_4BIT_PROG); // single line prog/read
+       // p_nand_info->feature &= (~FEA_4BIT_PROG); // single line prog, quad line read
        if (p_nand_info->feature & FEA_4BIT_READ) {
                if ((p_nand_info->has_qe_bits && sfc_nand_enable_QE() == SFC_OK) ||
                    !p_nand_info->has_qe_bits) {
```

rkflash single line flash patch:

```
diff --git a/drivers/rkflash/sfc_nor.c b/drivers/rkflash/sfc_nor.c
index c38801b4fb0d..4badff4249ac 100644
--- a/drivers/rkflash/sfc_nor.c
+++ b/drivers/rkflash/sfc_nor.c
@@ -678,7 +678,9 @@ static int snor_parse_flash_table(struct SFNOR_DEV *p_dev,
                        p_dev->write_status = snor_write_status1;
                else if (i == 2)
                        p_dev->write_status = snor_write_status2;

+               g_spi_flash_info->feature &= ~(FEA_4BIT_READ | FEA_4BIT_PROG); // single
line prog/read
+               // g_spi_flash_info->feature &= ~(FEA_4BIT_PROG); // single line prog, quad
line read
                if (g_spi_flash_info->feature & FEA_4BIT_READ) {
                        ret = SFC_OK;
                        if (g_spi_flash_info->QE_bits)
```

8.8 Abnormal upgrade or startup after replacing flash on the support list for Flash materials

Usually, when new flash is used, it is necessary to update the storage driver synchronously. Apply for Redmine permission from RK business and synchronize the storage patch.

8.9 When UBIFS is mounted with ro attribute, it is not a real read-only file system

When UBIFS is mounted with ro attribute, the file system still involves storage algorithms that involve Nand erase and write operations in the background. It is recommended to choose SquashFS and mount it under the UBI Block device to achieve a true read-only file system.

8.10 Troubleshooting Other Issues

If you have any other questions, it is recommended to read the document "Rockchip_Developer_FAQ_Storage_CN.pdf", Redmine path:

Rokchip_Developer_FAQ_Storage_CN.pdf

If you have some basic learning needs for flash, it is recommended to read the sections "Flash Introduction" and "Granule Verification" in the document "Rockchip_Application_Notes_Storage_CN.pdf", Redmine path:

Rockchip_Application_Notes_Storage_CN

If you have a need for dual storage driver development, it is recommended to read the document "Rockchip_Developer_Guide_Dual_Storage_CN.pdf", Redmine path:

Rockchip_Developer_Guide_Dual_Storage_CN.pdf

9. Reference documents

[1] UBI FAQ: http://www.linux-mtd.infradead.org/faq/ubi.html

[2] UBIFS FAQ: http://www.linux-mtd.infradead.org/faq/ubifs.html#L_lebsz_mismatch

[3] MTD FAQ: http://www.linux-mtd.infradead.org/faq/general.html